

Coworking Platform – MVP Software Development Plan

This document defines a **practical, build-ready software plan** for a Venture-X-style coworking platform, covering **web + mobile apps**, designed from both **Owner** and **Customer** perspectives. It is written so a dev team (or Cursor) can start implementation immediately.

1. Product Vision

Build a **multi-location coworking management platform** where: - Owners manage **locations, spaces, pricing, inventory, and admins** - Admins manage **day-to-day operations per location** - Customers **discover, book, and subscribe** to coworking spaces - Payments are **recurring, automated, and plan-based**

Primary goal of MVP:

💡 **Start collecting real payments for real spaces with minimal operational friction**

2. User Roles & Permissions

2.1 Roles

1 Platform Super Admin (internal)

- Approve owners
- View all locations & revenue
- Handle disputes, refunds, suspensions

2 Owner

- Can own **multiple locations**
- Can add **multiple admins**
- Full financial visibility

3 Admin

- Assigned to **one or more locations**
- Manages inventory, bookings, users
- Cannot see other locations' financials

4 Customer (Member)

- Books spaces
 - Subscribes to plans
 - Manages payment methods
-

3. Core Entities (Data Models)

3.1 Organization

- id
- name
- owner_id
- branding (logo, colors)

3.2 Location

- id
- organization_id
- name
- address
- timezone
- amenities
- status (active/inactive)

3.3 Space Type

- PRIVATE_OFFICE
- SHARED_DESK
- CONFERENCE_ROOM
- VIRTUAL_OFFICE

3.4 Space (Inventory Unit)

- id
- location_id
- space_type
- size_sqft
- capacity
- price_monthly
- price_daily (optional)
- availability_status
- floor_plan_id

3.5 Floor Plan

- id

- location_id
- image_url
- scale (optional)

3.6 Floor Plan Marker

- id
- floor_plan_id
- space_id
- x_coordinate
- y_coordinate
- width
- height

3.7 Subscription Plan

- id
- space_type
- billing_cycle (monthly / quarterly / 6mo / yearly)
- price
- access_rules

3.8 Booking

- id
- user_id
- space_id
- start_date
- end_date
- status

3.9 Payment

- id
- user_id
- subscription_id
- amount
- provider (Stripe)
- status

4. Owner Experience (Web – Priority)

4.1 Owner Dashboard (MVP)

Day 1 must-have: - Total revenue (MTD) - Active members - Occupancy by location - Failed payments alerts

4.2 Location Management

Owners can: - Create locations - Upload floor plans (PNG/JPG) - Define amenities - Assign admins

4.3 Floor Plan Editor (CRITICAL MVP FEATURE)

Functionality: - Upload floor plan image - Draw rectangles on image - Assign: - Space ID - Space type - Size - Price - Save spatial mapping

Tech suggestion: - Canvas-based editor (Konva.js / Fabric.js)

4.4 Inventory Management

- List of all spaces
 - Bulk price update
 - Enable/disable spaces
 - See assigned customers
-

4.5 Admin Management

- Invite admin by email
 - Assign location access
 - Permission levels (read / write)
-

5. Admin Experience (Web First)

Admins manage **operations per location**:

5.1 Admin Dashboard

- Today's bookings
- Available spaces
- Expiring subscriptions

5.2 Space Control

- Mark space as maintenance
- Reassign customer
- Adjust availability

5.3 Customer Management

- View members

- Manual booking (walk-ins)
 - Apply discounts (MVP optional)
-

6. Customer Experience (Web + Mobile)

6.1 Onboarding

- Sign up (email, Google, Apple)
 - Choose location
 - Choose space or plan
-

6.2 Browse & Select Space

- View floor plan (read-only)
 - See space details
 - Compare prices
-

6.3 Subscription & Payments

Supported billing: - Monthly - Quarterly - 6-month - Yearly

Payment flow: - Add credit card - Auto-renew subscription - Invoice generation

Tech: - Stripe Subscriptions - Stripe Customer Portal (MVP)

6.4 Customer Dashboard

- Active subscription
 - Upcoming invoices
 - Payment methods
 - Cancel / upgrade plan
-

7. MVP Scope (STRICT)



Must Have (Day 1 MVP)

- Owner creates location
- Upload floor plan
- Define spaces
- User subscribes

- Credit card charged
- Admin manages location

Not MVP (Phase 2)

- Mobile key access
 - IoT sensors
 - Dynamic pricing
 - Marketplace across locations
-

8. Tech Architecture

Frontend

- Web: React (Next.js recommended for SSR + routing)
- Mobile: React Native (Expo)

Backend

- API: FastAPI
- DB: PostgreSQL
- ORM: SQLAlchemy 2.0 (or SQLModel)
- Migrations: Alembic

Auth

- Auth.js / Clerk / Auth0 (email + Google + Apple)
- JWT access tokens + refresh tokens (or provider session)

Payments

- Stripe (Subscriptions + PaymentIntents + Webhooks)

Storage

- AWS S3 (floor plans + documents)

Infra (reference)

- AWS ECS/Fargate or EKS
 - RDS Postgres
 - CloudFront
-

8.1 Public IDs (Required)

 **Never expose database IDs in URLs.**

- Every major entity must have:
- `id` (internal DB primary key)
- `public_id` (UUIDv4) used in all APIs and URLs

Entities that must have `public_id`: - Organization, Location, FloorPlan, Space, Plan, Subscription, Booking, Payment, Discount, User

API examples: - `/api/locations/{location_public_id}` - `/api/spaces/{space_public_id}`

Implementation notes: - Add a `PublicIdMixin` with `public_id = uuid4()` - Unique index on `public_id` - Keep internal `id` for joins/performance

9. Security & Payments (MVP)

This product handles **payments and credit cards**, so the #1 rule is:

 **Never store or process raw credit card numbers yourself.**

Use Stripe to keep you in the lowest PCI scope (typically SAQ A). Stripe stores cards; your system stores only references.

9.1 Credit Card Safety (Must Have)

- Use **Stripe Elements** (web) and **Stripe Payment Sheet** (mobile) so card data goes directly to Stripe.
- Store only:
 - `stripe_customer_id`
 - `stripe_subscription_id`
 - `stripe_payment_intent_id`
 - (optional) `stripe_default_payment_method_id`
- Never store:
 - card number, CVV, full expiry, magnetic stripe data

9.2 Webhooks (Critical)

All payment truth comes from Stripe webhooks: - `invoice.paid` - `invoice.payment_failed` - `customer.subscription.updated` - `customer.subscription.deleted` - `payment_intent.succeeded` - `payment_intent.payment_failed`

Webhook requirements: - Verify Stripe signature - Idempotency + dedupe (event id unique) - Persist event log (`payment_events` table)

9.3 Data Protection

- HTTPS everywhere
- Secrets in a secret manager (AWS Secrets Manager)
- Signed URLs for S3 downloads
- Encrypt sensitive PII at rest (phone/address/docs)

9.4 Access Control + Auditing

- Strict RBAC (Owner/Admin/Customer)
- Location scoping for admins
- Audit logs for:
 - discounts/overrides
 - cancellations
 - membership changes

9.5 Fraud / Abuse Basics

- Rate limit auth + payment endpoints
 - Email verification
 - Stripe Radar defaults (good baseline)
-

Frontend

- Web: Next.js
- Mobile: React Native (Expo)

Backend

- API: FastAPI / NestJS
- DB: PostgreSQL
- Auth: Auth.js / Clerk

Payments

- Stripe (Subscriptions + Webhooks)

Storage

- AWS S3 (floor plans)

Infra

- AWS ECS / Fargate
 - CloudFront
 - RDS Postgres
-

9. Security, Payments & Compliance (VERY IMPORTANT)

This section covers **public UUID usage**, **credit card safety**, and **things founders usually miss**.

9.1 Public IDs (CONFIRMED)

 **Rule: NEVER expose internal DB IDs**

Implementation standard: - Every externally exposed entity uses a `public_id` (UUID v7 or UUID v4) - Internal numeric IDs are **DB-only**

Entities that MUST have public_id: - Organization - Location - Space - FloorPlan - Subscription - Booking - Payment - User

API rule:

```
/api/locations/{public_id}
/api/spaces/{public_id}
/api/subscriptions/{public_id}
```

DB rule: - `id` = internal (INT / BIGINT) - `public_id` = UUID, indexed, unique

9.2 Credit Card & Payment Security (CRITICAL)

 **NEVER store credit card numbers** — ever.

Correct approach (MVP & future-proof): - Use **Stripe Elements** or **Stripe Payment Sheet** - Card data goes **directly to Stripe** - Your backend only stores: - `stripe_customer_id` - `stripe_payment_method_id` (reference only)

Result: - You are **PCI SAQ-A compliant** (lowest burden)

9.3 Payment Flow Architecture

Subscriptions: 1. Frontend → create Stripe customer 2. Stripe handles card entry 3. Backend stores customer reference 4. Stripe webhooks control state: - `invoice.paid` → activate subscription - `invoice.payment_failed` → mark past_due

Bookings (hourly/day): - Stripe PaymentIntent (one-time) - Capture before confirming booking

9.4 Webhooks (Non-negotiable)

Must handle these events: - customer.subscription.created - customer.subscription.updated - invoice.paid - invoice.payment_failed - payment_intent.succeeded - payment_intent.payment_failed

Rules: - Webhooks are the **source of truth**, not frontend - Verify Stripe signature on every webhook

9.5 Authorization & Abuse Prevention

- Role-based access control (Owner / Admin / Customer)
 - Object-level checks (admin can only access assigned locations)
 - Rate limiting on:
 - booking creation
 - payment intents
-

9.6 Audit Logs (You WILL need this)

Track: - Admin price overrides - Discount creation - Space reassignment - Manual booking changes

Audit log fields: - actor_id - action - entity_type - entity_public_id - before_state - after_state - timestamp

9.7 Things Founders Commonly Forget (You Should Not)

-  Failed payments retry logic (Stripe Smart Retries)
 -  Grace period rules: - Example: 3 days past_due → restrict access
 -  Refund rules: - Booking refund vs subscription proration
 -  Timezone correctness (location timezone ALWAYS wins)
 -  Idempotency keys for: - booking creation - payment intents
 -  Data retention: - Keep invoices forever - Soft-delete users
-

9.8 Future Compliance (Not MVP, But Design For It)

- GDPR / data export
 - SOC2 (if selling to enterprises)
 - Multi-currency + tax engines
-

10. Confirmed Product Decisions (Locked)

Based on owner input, the following decisions are **FINAL for MVP**:

Subscription Model

- Customers subscribe to a **specific Space ID (office/desk)**
- Space is reserved exclusively for the subscription duration

Multi-Location Access

- A single subscription **can grant access to multiple locations**
- Access rules will be controlled via plan configuration

Admin Power

- Admins **can override pricing**
- Admins **can apply discounts** (one-time or recurring)
- All overrides must be audit-logged

Bookings vs Subscriptions

- MVP supports **both**:
- Recurring subscriptions (primary revenue)
- Hourly / daily bookings (conference rooms, shared desks)

Market & Currency

- MVP launches with **single country + single currency**
 - Architecture will be **multi-currency ready** for Phase 2
-

11. Stripe Billing Architecture (Updated)

Subscription Billing

- Stripe Subscriptions tied to:
- Customer
- Space ID
- Billing cycle (monthly / quarterly / 6mo / yearly)

One-Time Bookings

- Stripe Payment Intents
- Optional invoice generation

Discounts

- Stripe Coupons (global)
- Manual price overrides (stored internally)

Required Webhooks

- invoice.paid
 - invoice.payment_failed
 - customer.subscription.updated
 - customer.subscription.deleted
-

11. Recommended Build Timeline

- Week 1–2: Auth + Org + Location
 - Week 3–4: Floor plan editor
 - Week 5–6: Subscriptions & payments
 - Week 7–8: Admin + Customer dashboards
-

Outcome

At MVP launch, you will: - Onboard real owners - Upload real floor plans - Sell real subscriptions - Collect recurring revenue

12. Confirmed MVP Decisions (Your Answers)

Subscription model

- Customers subscribe to a **specific Space ID** (a specific private office/desk).
- This means **inventory must prevent double-selling** the same space for overlapping time.

Multi-location access

- A customer can access **multiple locations under one subscription**.
- MVP approach: treat the subscription as having a **Home Space (primary)** plus **Location Access** entries for other locations.
- Example: Primary private office at Location A; coworking/day-pass style access at Locations B and C.

Admin pricing power

- Admins can **override price / apply discounts**.
- MVP requirements:
 - Require a **reason** for every override
 - Log who did it + when (audit trail)

- Optional: permission toggle per admin

Bookings vs subscriptions

- MVP includes **hourly/day bookings** in addition to subscriptions.
- Billing:
- Subscriptions → Stripe Subscriptions
- Bookings → Stripe one-time payments (PaymentIntents)

Currency

- Launch with **single country & single currency**.
- Design for future:
- Add `currency` on Location
- Store money as integer cents + currency code
- Stripe account/price strategy can expand later

If you want, next I can: - Convert this into **Cursor-ready tasks** - Design **DB schema diagrams** - Create **API contracts (OpenAPI)** - Break into **Jira tickets**

UI Design System & QA Plan (Option A: Charcoal + Indigo/Blue)

This section defines the **UI theme spec + implementation tokens + test plan** so the UI stays **modern, consistent, and premium** across Web + Mobile.

13. UI Theme Goals

Vibe: Modern • Calm • Premium • Professional • Trustworthy

Core rule: - 80–85% neutral surfaces - 10–15% accent - Minimal borders, clean spacing, consistent typography

14. Design Tokens (Source of Truth)

14.1 Colors (Light Theme)

Neutrals - Background (app): `#F9FAFB` - Surface / Card: `#FFFFFF` - Surface 2 (subtle): `#F3F4F6` -
Border: `#E5E7EB` - Divider: `#E5E7EB`

Text - Text Primary: #111827 (dark charcoal) - Text Secondary: #6B7280 - Text Muted: #9CA3AF

Accent (choose one and use everywhere) - Indigo: #4F46E5 (recommended) - Hover/Pressed: #4338CA - Accent subtle bg: #EEF2FF

Status - Success: #16A34A - Warning: #D97706 - Error: #DC2626 - Info: #2563EB

14.2 Typography

- Font family: **Inter**
- Base size: 16px (web), 15–16 (mobile)
- Weights:
- Headings: 600–700
- Body: 400–500
- Labels: 500

14.3 Spacing & Layout

- Spacing scale: 4, 8, 12, 16, 24, 32, 48
- Grid: 12-col (web)
- Container max width: 1200–1280px
- Card padding: 16–24px

14.4 Radius & Shadows

- Radius:
- Buttons/inputs: 10px
- Cards: 14px
- Modals: 16px
- Shadow: subtle only
- Card shadow: small soft (avoid heavy drop shadows)

14.5 Buttons

Primary CTA - Background: Accent Indigo - Text: White - Hover: Accent Hover - Height: 44px

Secondary - Background: White - Border: Border color - Text: Primary text

Destructive - Background: Error - Text: White

14.6 Forms

- Input height: 44px
 - Border: #E5E7EB
 - Focus ring: Accent at 20–30% opacity
 - Error state: border error + helper text error
-

15. UI Modules to Build in V1 (Screens + Components)

15.1 Core Reusable Components (Design System)

Build once and reuse everywhere.

Atoms - Button (Primary/Secondary/Ghost/Destructive) - Input, TextArea - Select/Dropdown - Checkbox/Radio - Badge (status) - Tooltip - Spinner/Loader - Icon wrapper

Molecules - FormField (label + input + helper + error) - SearchBar - EmptyState - Toast notifications - ConfirmDialog

Organisms - AppShell (sidebar/topbar) - DataTable (pagination, sorting) - CardList (mobile-friendly alternative to tables) - Modal (standard sizes)

15.2 Web Screens (Owner/Admin)

Owner - Login / Signup - Owner Dashboard - Organization Settings - Locations List - Location Detail (tabs) - Overview - Floor Plans - Spaces Inventory - Admins - Pricing - Members - Payments/Invoices

Admin - Admin Dashboard (per location) - Bookings calendar/list - Create booking (walk-in) - Manage spaces (availability/maintenance) - Manage members - Apply discount/override price

15.3 Customer Screens (Web + Mobile)

- Login / Signup
- Browse locations
- Location detail + floor plan (read-only)
- Space detail (price, size, type, availability)
- Subscribe to a Space
- Book hourly/day
- Payments (add card / invoices)
- Account settings

16. Floor Plan UI Requirements (Modern + Clean)

16.1 Owner/Admin Editor

- Canvas editor with pan/zoom
- Draw/resize rectangles
- Assign Space details (ID, type, size, capacity, pricing)
- Color overlay rules:
 - Neutral overlays only (avoid rainbow)
 - AVAILABLE = subtle accent outline
 - OCCUPIED = gray fill

- MAINTENANCE = warning tint

16.2 Customer Viewer

- Read-only
 - Tap space → details bottom sheet
 - Clear CTA for Subscribe/Book
-

17. UI Acceptance Criteria (What “Modern” Means)

These are **pass/fail rules**.

- 1) **Accent usage** - Only ONE accent color used across CTAs, focus rings, links.
 - 2) **Density** - No page should feel cramped. - Minimum padding in cards: 16px.
 - 3) **Typography** - Consistent font sizes. - No more than 3 heading sizes used per page.
 - 4) **Consistency** - All modals use the same layout: - Title - Content - Footer actions
 - 5) **Accessibility** - Color contrast meets WCAG AA for primary text. - Keyboard focus visible on web.
-

UI Test Plan (Functional + Visual)

18. Testing Strategy Overview

You need 3 layers of UI testing: 1) **Component tests** (fast) 2) **E2E tests** (flows) 3) **Visual regression tests** (detect UI drift)

19. Tooling Recommendations

Web

- Component: Storybook + Testing Library
- E2E: Playwright
- Visual regression: Chromatic (Storybook) OR Playwright screenshots

Mobile

- UI automation: Detox (recommended) or Appium
 - Visual: screenshot testing (Detox snapshots)
-

20. UI Test Cases (V1)

20.1 Global UI Tests

TC-UI-001 Theme Tokens Applied - Steps: Open any page → verify background, card, border, text colors match tokens. - Expected: No off-brand colors. Only neutrals + accent + status colors.

TC-UI-002 Typography Consistency - Steps: Inspect headings/body/labels on Dashboard + Forms. - Expected: Inter used everywhere, weights consistent.

TC-UI-003 Spacing & Density - Steps: Check cards, modals, forms. - Expected: Card padding $\geq 16\text{px}$; button height 44px; no overflowing content.

TC-UI-004 Dark Text Contrast - Steps: Check key pages with normal brightness. - Expected: Primary text readable; no light-gray-on-white for important content.

20.2 Auth Screens

TC-AUTH-001 Login Layout - Steps: Open login. - Expected: Clean centered card, primary CTA accent, secondary actions subtle.

TC-AUTH-002 Validation Errors - Steps: Submit empty form. - Expected: Inputs red border + helper text; no layout jump.

20.3 Owner Location Setup

TC-OWNER-LOC-001 Create Location Form - Steps: Create location. - Expected: Proper form field spacing; save CTA uses accent.

TC-OWNER-LOC-002 Assign Admin - Steps: Invite admin, assign locations. - Expected: Table/cards consistent; toasts appear on success.

20.4 Floor Plan Editor

TC-FP-001 Upload Floor Plan - Steps: Upload image. - Expected: Image renders crisp; loader shown while uploading.

TC-FP-002 Draw Space - Steps: Draw rectangle. - Expected: Rectangle snaps cleanly; handles visible; details modal opens.

TC-FP-003 Save Marker - Steps: Assign space ID/type/size/price → save. - Expected: Marker persists after refresh.

TC-FP-004 Visual Clarity - Steps: Create multiple spaces. - Expected: Overlays do not create noisy colors; labels readable.

20.5 Customer Subscription Flow

TC-SUB-001 Subscribe CTA - Steps: Open space detail → click Subscribe. - Expected: Primary CTA accent; clear pricing & billing cycle.

TC-SUB-002 Stripe Payment UI - Steps: Add card (web/mobile). - Expected: Card entry handled by Stripe UI components, not custom inputs.

TC-SUB-003 Success State - Steps: Complete payment. - Expected: Success page shows subscription status + invoice link.

TC-SUB-004 Failure State - Steps: Use failing payment method. - Expected: Friendly error, retry CTA, no duplicate subscription records.

20.6 Booking Flow (Hourly/Day)

TC-BOOK-001 Select Date/Time UI - Steps: Choose time range. - Expected: Time picker consistent; no overlaps allowed.

TC-BOOK-002 Payment Success - Steps: Pay for booking. - Expected: Booking becomes CONFIRMED only after payment succeeded webhook.

20.7 Admin Price Override

TC-OVR-001 Apply Discount - Steps: Admin overrides price. - Expected: Requires reason; shows discounted amount; audit log created.

TC-OVR-002 Permissions - Steps: Admin without pricing permission tries override. - Expected: Action blocked; UI hides controls.

21. Visual Regression “UI Snapshot” Plan

21.1 Pages to Snapshot (Web)

Take baseline screenshots and compare on every PR: - Login - Owner Dashboard - Location Detail - Floor Plan Editor - Space Detail - Subscribe Modal - Booking Flow

21.2 Component Stories to Snapshot (Storybook)

- Buttons (all variants)
- Inputs (default/focus/error/disabled)
- Cards
- Modals
- Badges
- Tables
- Toasts

Pass criteria: - No unexpected pixel diffs unless explicitly approved.

22. UI Review Checklist (Manual QA)

Before release, check: - CTA buttons are accent color everywhere - No random blues/greens outside tokens - Consistent spacing and alignment - Form errors look consistent - Loading states exist (no blank screens) - Empty states exist (no dead ends) - Mobile tap targets $\geq 44\text{px}$

23. Dev Implementation Notes (To Keep UI Consistent)

- Create a `theme.ts` / `tokens.ts` file as the single source of truth.
 - Use a component library approach:
 - Web: Tailwind + headless components OR shadcn/ui patterns
 - Mobile: RN components that map to the same tokens
 - Do not hardcode colors in pages—use tokens only.
-

24. Deliverables for V1 UI

1) `tokens.ts` (colors/spacing/radius/typography) 2) Component library (Buttons, Inputs, Modal, Card, Table) 3) Storybook (web) 4) Playwright E2E tests (core flows) 5) Visual regression pipeline (Chromatic or screenshots)
